DOCUMENT 10 — AUTHENTICATION & AUTHORIZATION SPECIFICATION

1. Dokümanın Amacı

Smart Chef & Pantry AI uygulamasında kullanıcı kimlik doğrulama (Authentication) ve yetkilendirme (Authorization) süreçlerinin güvenli, merkezi ve ölçeklenebilir şekilde yönetilmesi bu dokümanın temel amacıdır.

Uygulamada kullanıcıların iki farklı kullanım durumu olacaktır:

1. Guest User
2. Authenticated User

Buna ek olarak, tarif ve ingredient verilerinin yönetimi için Admin rolü bulunacaktır.

Authentication ve Authorization sistemi aşağıdaki temel yapıya dayanacaktır:

Flutter Application
↓
Google Social Login
↓
Backend Social Token Verification
↓
User Find / Create
↓
Backend JWT
↓
Protected API Requests
↓
Authorization

Authentication işlemi kullanıcının kim olduğunu doğrularken, Authorization kullanıcının hangi işlemleri gerçekleştirebileceğini belirleyecektir.

---

2. Authentication ve Authorization Ayrımı

Authentication:

Kullanıcının kim olduğunu doğrulama işlemidir.

Authorization:

> Doğrulanmış kullanıcının hangi kaynaklara ve işlemlere erişebileceğini belirleme işlemidir.

Örneğin:

Google Login
↓
Authentication
↓
User authenticated

sonrasında:

GET /api/favorites

isteği için kullanıcının authenticated olması gerekir.

Admin endpoint'i için ise:

Authenticated User
+
Admin Role

koşulu aranır.

---

3. Kullanıcı Rolleri

MVP kapsamında üç farklı erişim seviyesi tanımlanacaktır.

3.1. Guest User

Uygulamaya giriş yapmamış kullanıcıdır.

Guest kullanıcı aşağıdaki işlemleri gerçekleştirebilir:

- ingredient listesini görüntüleme
- ingredient arama
- ingredient seçme
- recipe recommendation alma
- recipe sonuçlarını görüntüleme
- recipe detail görüntüleme
- recipe source URL açma

- YouTube video açma

Guest kullanıcı aşağıdaki işlemleri gerçekleştiremez:

- favorite ekleme
- favorite kaldırma
- favorites listesini görüntüleme
- persistent cart kullanımı
- profile görüntüleme
- logout

Protected bir işlem gerçekleştirmek istediğinde kullanıcı login ekranına yönlendirilir.

3.2. Registered User

Google Social Login ile authentication işlemini tamamlamış kullanıcıdır.

Registered user:

- Guest kullanıcı işlemlerinin tamamını
- favorite ekleme
- favorite kaldırma
- favorites görüntüleme
- cart görüntüleme
- cart item ekleme
- cart quantity değiştirme
- cart item silme
- cart temizleme
- profile görüntüleme
- logout

işlemlerini gerçekleştirebilir.

3.3. Admin

Admin, recipe ve ingredient verilerini yönetme yetkisine sahip authenticated kullanıcıdır.

Admin:

- recipe oluşturabilir
- recipe güncelleyebilir
- recipe silebilir/pasifleştirebilir
- ingredient oluşturabilir
- ingredient güncelleyebilir
- ingredient silebilir/pasifleştirebilir

Admin işlemleri mobil uygulamadaki normal kullanıcı işlemlerinden ayrı tutulacaktır.

MVP'de ayrı bir admin mobil uygulaması geliştirilmesi zorunlu değildir.

---

4. Authentication Provider

MVP authentication provider olarak Google kullanılacaktır.

Temel login akışı:

Flutter
↓
Google Sign-In
↓
Google ID Token
↓
POST /api/auth/social
↓
Flask Backend
↓
Google Token Verification
↓
Find / Create User
↓
Generate Backend JWT
↓
Return JWT
↓
Flutter

Backend Google token'ını kendisi doğrulamalıdır.

Flutter tarafından gönderilen kullanıcı bilgileri tek başına güvenilir authentication kanıtı olarak kabul edilmemelidir.

---

5. Social Login Request

Endpoint:

POST /api/auth/social

Request:

```
{
  "provider": "google",
  "id_token": "GOOGLE_ID_TOKEN"
}
```

Backend:

1. provider bilgisini kontrol eder.
2. ID token formatını doğrular.
3. Google token'ını doğrular.
4. Token'ın geçerlilik süresini kontrol eder.
5. Token'ın uygulamaya ait olup olmadığını kontrol eder.
6. Google kullanıcı bilgilerini elde eder.
7. Database'de kullanıcıyı arar.
8. Kullanıcı yoksa oluşturur.
9. Backend JWT üretir.
10. JWT ve kullanıcı bilgilerini Flutter'a gönderir.

---

6. Social Token Verification

Backend, Flutter tarafından gönderilen `id_token` değerine güvenerek doğrudan kullanıcı oluşturmayacaktır.

Aşağıdaki bilgiler doğrulanmalıdır:

Token Signature
Issuer
Audience
Expiration
Subject
Email

Özellikle `audience` kontrolü önemlidir.

Token'ın gerçekten uygulamanın tanımlı Google Client ID'sine ait olması gerekir.

Temel mantık:

Received ID Token
↓
Verify Signature
↓
Verify Issuer
↓

Verify Audience
↓
Verify Expiration
↓
Extract User Identity

Doğrulama başarısız olursa:

SOCIAL_AUTH_FAILED

hatası döndürülür.

---

7. User Creation / Find Process

Google authentication başarılı olduktan sonra backend database üzerinde kullanıcıyı kontrol eder.

Temel arama:

provider = google
provider_user_id = Google subject

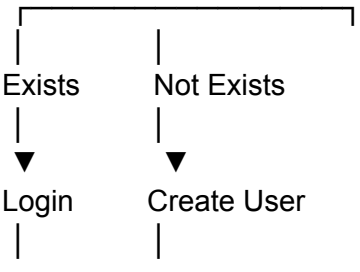
kombinasyonu üzerinden yapılacaktır.

Kullanıcı bulunursa mevcut kullanıcı kullanılacaktır.

Kullanıcı bulunamazsa yeni kullanıcı oluşturulacaktır.

Örnek:

Google User
↓
provider = google
provider_user_id = Google subject
↓
Database Search
↓





---

8. User Database Structure

`users` tablosunda authentication için aşağıdaki alanlar kullanılacaktır:

users
---
id
provider
provider_user_id
name
email
profile_image_url
created_at
updated_at

Örneğin:

```
{
  "id": "uuid-user-001",
  "provider": "google",
  "provider_user_id": "google-user-id",
  "name": "Esma",
  "email": "user@example.com",
  "profile_image_url": "https://...",
  "created_at": "2026-01-01T12:00:00Z",
  "updated_at": "2026-01-01T12:00:00Z"
}
```

`provider + provider_user_id` kombinasyonu unique olmalıdır.

---

9. Backend JWT

Google token ile backend authentication işlemi tamamlandıktan sonra backend kendi authentication token'ını üretmelidir.

Bu token:

Backend JWT

olacaktır.

Flutter sonraki protected API isteklerinde Google ID token yerine backend JWT kullanacaktır.

Akış:

Google ID Token
↓
Backend Verification
↓
Backend JWT
↓
Flutter
↓
Protected API Requests

Bu ayrım backend authentication mimarisinin Google'a doğrudan bağımlı olmasını azaltır.

---

10. JWT İçeriği

JWT içerisinde minimum gerekli kullanıcı kimliği bilgileri bulunmalıdır.

Örneğin:

```
{
  "sub": "user-uuid",
  "role": "user"
}
```

Admin için:

```
{
  "sub": "admin-user-uuid",
  "role": "admin"
}
```

JWT içerisine gereksiz kişisel bilgiler konulmamalıdır.

Özellikle:

- password

- sensitive personal data
- unnecessary profile information

JWT içerisine yazılmamalıdır.

---

11. JWT Secret Management

JWT imzalama anahtarı source code içerisine yazılmamalıdır.

Environment variable kullanılacaktır.

Örneğin:

JWT_SECRET_KEY=...

`.env` dosyasında tutulacaktır.

`.env`:

JWT_SECRET_KEY=super-secret-value

şeklinde olabilir.

Ancak gerçek secret değerleri Git repository'sine gönderilmemelidir.

`.gitignore` içerisinde:

.env

bulunmalıdır.

`.env.example` dosyasında yalnızca örnek değişken isimleri bulunmalıdır.

Örneğin:

JWT_SECRET_KEY=
DATABASE_URL=
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=

---

12. JWT Expiration

JWT sonsuza kadar geçerli olmamalıdır.

Access token için expiration süresi belirlenmelidir.

Örneğin:

JWT_ACCESS_TOKEN_EXPIRES

configuration üzerinden yönetilebilir.

MVP'de refresh token mekanizması zorunlu değildir.

İlerleyen sürümlerde:

Access Token
+
Refresh Token

mimarisi değerlendirilebilir.

---

13. Flutter Token Storage

Backend JWT Flutter tarafında güvenli bir storage mekanizmasıyla saklanmalıdır.

Token düz bir şekilde:

SharedPreferences

içerisinde tutulmamalıdır.

MVP'de platformun güvenli credential storage mekanizması kullanılmalıdır.

Örneğin:

Flutter
↓
Secure Storage
↓
Access Token

Token:

- login sonrasında kaydedilir
- logout sırasında silinir

- API request sırasında alınır
- expiration durumunda authentication state güncellenir

---

14. Dio Authentication Interceptor

Flutter'da API isteklerinde JWT'nin otomatik olarak eklenmesi için Dio interceptor kullanılacaktır.

Akış:

Flutter API Request
↓
Dio
↓
Auth Interceptor
↓
Read JWT from Secure Storage
↓
Add Authorization Header
↓
Backend

Header:

Authorization: Bearer <JWT>

şeklinde gönderilir.

UI içerisinde her API çağrısında token'ın manuel olarak eklenmesi tercih edilmeyecektir.

---

15. Protected Endpoint

Authentication gerektiren endpoint'lerde backend JWT kontrolü yapılacaktır.

Örneğin:

GET /api/favorites

isteği authenticated user gerektirir.

Backend akışı:

Request
  ↓
Authorization Header
  ↓
JWT Validation
  ↓
Token Valid?
  ├── No → 401
  │
  └── Yes
      ↓
   Extract User ID
      ↓
    Service

---

16. Current User ID

Protected endpoint'lerde kullanıcı ID'si request body üzerinden alınmamalıdır.

Yanlış yaklaşım:

```
{
  "user_id": "some-user-id",
  "recipe_id": "recipe-001"
}
```

Backend'in gönderilen `user_id` değerine güvenmesi güvenlik açığı oluşturabilir.

Doğru yaklaşım:

JWT
 ↓
sub
 ↓
current_user_id
 ↓
Service

Örneğin:

POST /api/favorites

Body:
{

```
    "recipe_id": "recipe-001"
}
```

Backend kullanıcıyı JWT'den belirler.

JWT
sub = user-001

Böylece favorite:

user-001
+
recipe-001

olarak oluşturulur.

---

17. Authorization Kontrolü

Authentication başarılı olması kullanıcının her endpoint'e erişebileceği anlamına gelmez.

Örneğin:

Authenticated User

aşağıdaki endpoint'e erişemez:

POST /api/admin/recipes

Çünkü bu endpoint:

Admin Role

gerektirir.

Kontrol:

```
JWT Valid?
  ↓
Role = admin?
  ↓
Yes → Continue
No  → 403 Forbidden
```

---

18. Admin Authorization

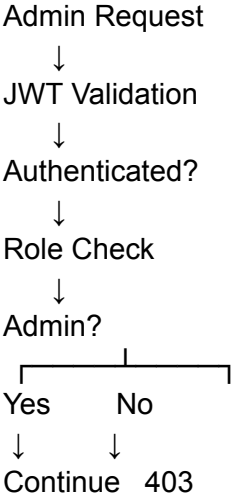
Admin kontrolü backend'de yapılacaktır.

Örneğin decorator:

@admin_required

şeklinde kullanılabilir.

Akış:



Admin yetkisi yalnızca Flutter tarafında kontrol edilmemelidir.

Flutter'ın:

isAdmin = true

demesi backend açısından yeterli değildir.

Gerçek authorization backend tarafından yapılmalıdır.

---

19. Guest → Login → Previous Context

Guest kullanıcı protected bir işlem yapmaya çalıştığında login ekranına yönlendirilmelidir.

Örneğin:

Recipe Detail
↓
Favorite Button
↓
User not authenticated
↓
Login Screen
↓
Google Login
↓
Backend JWT
↓
Return to Recipe Detail
↓
Favorite Action

Bu akış kullanıcı deneyimi açısından korunmalıdır.

Örneğin kullanıcı Recipe Detail ekranında favorite butonuna bastığında login olduktan sonra tekrar Home ekranına gönderilmesi yerine mümkün olduğunca önceki context korunmalıdır.

---

20. Authentication State

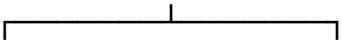
Flutter tarafında authentication state merkezi olarak yönetilecektir.

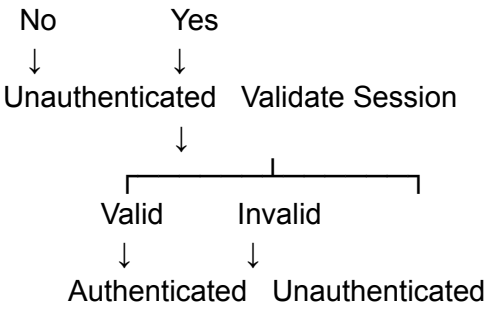
Önerilen state:

AuthState

Unauthenticated
Authenticating
Authenticated
Error

Örnek akış:

Application Start
↓
Check Secure Storage
↓
Token Exists?


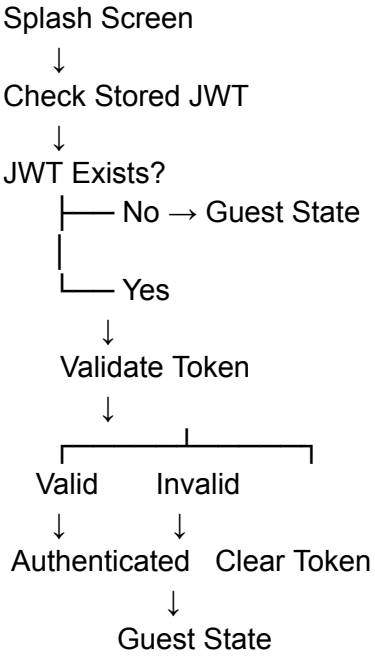


Bu state Riverpod üzerinden yönetilebilir.

---

21. Application Startup

Uygulama açıldığında:



Bu süreç tamamlanmadan kullanıcıya yanlış authentication state gösterilmemelidir.

---

22. `/api/me`

Authenticated kullanıcının mevcut session bilgisini kontrol etmek için:

GET /api/me

endpoint'i kullanılacaktır.

Request:

Authorization: Bearer <JWT>

Response:

```
{
  "success": true,
  "data": {
    "id": "uuid-user",
    "name": "Esma",
    "email": "user@example.com",
    "profile_image_url": "https://..."
  }
}
```

Bu endpoint Flutter tarafından application startup veya gerekli authentication kontrollerinde kullanılabilir.

---

23. Logout

Logout endpoint'i:

POST /api/auth/logout

olacaktır.

MVP'de JWT stateless authentication kullanılacağı için logout işleminin temel sorumluluğu Flutter tarafında token'ı güvenli storage'dan silmektir.

Akış:

```
User taps Logout
      ↓
POST /api/auth/logout
      ↓
Clear Secure Storage
      ↓
Clear Auth State
      ↓
Navigate to Guest State
```

MVP'de access token blacklist sistemi zorunlu değildir.

İleride refresh token veya token revocation mekanizması eklenebilir.

---

24. Token Expiration

API isteği sırasında backend:

TOKEN_EXPIRED

döndürürse Flutter:

1. mevcut token'ı temizlemeli
2. authentication state'i unauthenticated yapmalı
3. kullanıcıyı uygun şekilde login ekranına yönlendirmelidir

Akış:

API Request
↓
JWT Expired
↓
401 TOKEN_EXPIRED
↓
Clear Token
↓
Unauthenticated
↓
Login if Protected Action

---

25. Error Codes

Authentication ve authorization için aşağıdaki error code'lar kullanılacaktır:

AUTH_REQUIRED
INVALID_TOKEN
TOKEN_EXPIRED
SOCIAL_AUTH_FAILED
USER_NOT_FOUND
INVALID_PROVIDER
FORBIDDEN
ADMIN_REQUIRED

Örnek:

```
{
  "success": false,
  "error": {
    "code": "AUTH_REQUIRED",
    "message": "Authentication is required.",
    "details": {}
  }
}
```

Admin endpoint'inde:

```
{
  "success": false,
  "error": {
    "code": "ADMIN_REQUIRED",
    "message": "Admin access is required.",
    "details": {}
  }
}
```

---

26. Authentication API Endpoints

POST `/api/auth/social`

Amaç:

Google authentication işlemini tamamlamak ve backend JWT üretmek.

Access:

Public

Request:

```
{
  "provider": "google",
  "id_token": "GOOGLE_ID_TOKEN"
}
```

Response:

```
{
  "success": true,
  "data": {
    "access_token": "BACKEND_JWT",
    "token_type": "Bearer",
    "user": {
      "id": "uuid",
      "name": "User Name",
      "email": "user@example.com",
      "profile_image_url": "https://..."
    }
  }
}
```

GET `/api/me`

Amaç:

Mevcut authenticated kullanıcının bilgilerini almak.

Access:

Authenticated

POST `/api/auth/logout`

Amaç:

Logout işlemini gerçekleştirmek.

Access:

Authenticated

MVP'de temel olarak client-side token temizleme işlemi gerçekleştirilecektir.

---

27. Protected API Endpoint Matrix

Endpoint	Guest	User	Admin

GET `/api/ingredients`	✓	✓	✓
POST `/api/recipes/recommend`	✓	✓	✓
GET `/api/recipes/{id}`	✓	✓	✓
GET `/api/favorites`	✗	✓	✓
POST `/api/favorites`	✗	✓	✓
DELETE `/api/favorites/{id}`	✗	✓	✓
GET `/api/cart`	✗	✓	✓
POST `/api/cart/items`	✗	✓	✓
PATCH `/api/cart/items/{id}`	✗	✓	✓
DELETE `/api/cart/items/{id}`	✗	✓	✓
DELETE `/api/cart`	✗	✓	✓
POST `/api/admin/recipes`	✗	✗	✓
PUT `/api/admin/recipes/{id}`	✗	✗	✓
DELETE `/api/admin/recipes/{id}`	✗	✗	✓
POST `/api/admin/ingredients`	✗	✗	✓
PUT `/api/admin/ingredients/{id}`	✗	✗	✓
DELETE `/api/admin/ingredients/{id}`	✗	✗	✓

---

28. Security Rules

Authentication sisteminde aşağıdaki güvenlik kuralları uygulanacaktır:

28.1. Secret değerler source code içine yazılmamalıdır.

Kullanılacak değerler:

DATABASE_URL
JWT_SECRET_KEY

GOOGLE_CLIENT_ID
GOOGLE_CLIENT_SECRET

environment variable olarak tutulmalıdır.

28.2. Google token backend tarafından doğrulanmalıdır.

Flutter'dan gelen kullanıcı bilgilerine doğrudan güvenilmemelidir.

28.3. JWT client tarafından oluşturulmamalıdır.

JWT yalnızca backend tarafından oluşturulmalıdır.

28.4. Protected endpoint'lerde user ID request body'den alınmamalıdır.

Kullanıcı kimliği JWT üzerinden belirlenmelidir.

28.5. Admin kontrolü backend'de yapılmalıdır.

Flutter tarafındaki role bilgisi authorization için yeterli değildir.

28.6. Token loglanmamalıdır.

JWT, Google ID token veya secret değerleri application loglarında açık şekilde tutulmamalıdır.

28.7. HTTPS kullanılmalıdır.

Production ortamında authentication tokenlarının HTTP üzerinden taşınmasına izin verilmemelidir.

---

29. Backend Folder Responsibilities

Authentication ile ilgili dosyalar:

```
backend/
└── app/
    ├── routes/
    │   └── auth_routes.py
    │
    ├── services/
    │   └── auth_service.py
    │
    ├── schemas/
    │   └── auth.py
```

```
│
├── extensions/
│   └── jwt.py
│
├── utils/
│   └── decorators.py
│
├── errors/
│   ├── exceptions.py
│   └── handlers.py
│
└── models/
    └── user.py
```

Sorumluluk dağılımı:

auth_routes.py
↓
HTTP request/response

auth.py
↓
Request validation

auth_service.py
↓
Authentication business logic

jwt.py
↓
JWT configuration

decorators.py
↓
Authorization checks

user.py
↓
User database model

---

30. Flutter Authentication Structure

Flutter tarafında authentication feature:

lib/

```
└── features/
    └── auth/
        ├── data/
        │   ├── datasources/
        │   │   └── auth_remote_datasource.dart
        │   ├── models/
        │   │   └── user_model.dart
        │   └── repositories/
        │       └── auth_repository_impl.dart
        │
        ├── domain/
        │   ├── entities/
        │   │   └── user.dart
        │   ├── repositories/
        │   │   └── auth_repository.dart
        │   └── usecases/
        │       ├── login_with_google.dart
        │       ├── logout.dart
        │       └── get_current_user.dart
        │
        └── presentation/
            ├── pages/
            │   └── login_page.dart
            ├── providers/
            │   └── auth_provider.dart
            └── widgets/
                └── google_login_button.dart
```

Bu yapı Document 07'de belirlenen Flutter Clean Architecture ile uyumludur.

---

31. Authentication Flow

31.1. Successful Login

User
↓
Login Screen
↓
Google Sign-In
↓
Google ID Token
↓
POST /api/auth/social
↓
Backend verifies token

↓
Find/Create User
↓
Generate JWT
↓
Return JWT
↓
Secure Storage
↓
Authenticated State
↓
Home / Previous Context

31.2. Failed Login

Google Sign-In
↓
Backend
↓
Token Verification Failed
↓
SOCIAL_AUTH_FAILED
↓
Flutter Error State
↓
Show Login Error

31.3. Protected Action by Guest

Guest
↓
Recipe Detail
↓
Favorite
↓
Authentication Required
↓
Login Screen
↓
Google Login
↓
JWT
↓
Return to Recipe Detail
↓

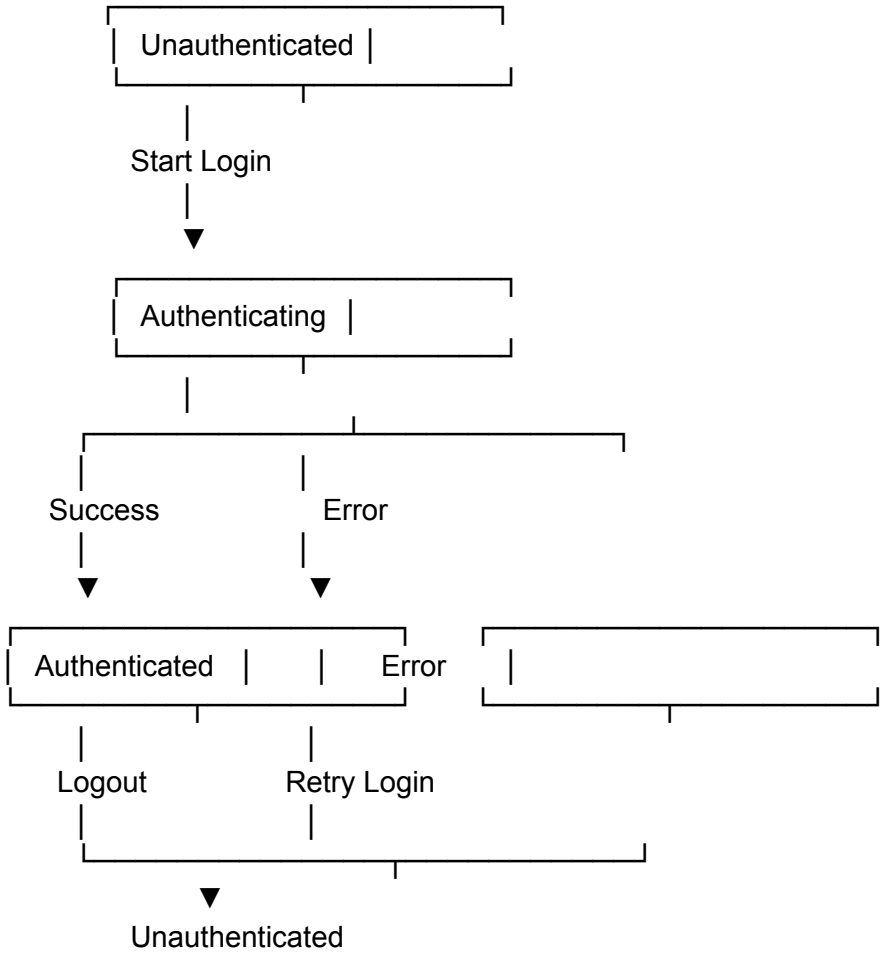
Favorite Request

31.4. Admin Request

Admin
↓
Admin API
↓
JWT Validation
↓
Role Check
↓
Admin?
├── Yes → Execute
└── No → 403 Forbidden

---

32. Authentication State Machine



---

33. Authorization Architecture

Authorization backend'de merkezi olarak gerçekleştirilecektir.

Temel yapı:

Request
↓
JWT Middleware
↓
Authentication
↓
Current User
↓
Authorization
↓
Role / Ownership Check
↓
Service

Bazı işlemlerde yalnızca role değil, resource ownership kontrolüne de ihtiyaç vardır.

Örneğin kullanıcı:

DELETE /api/favorites/recipe-001

isteği yaptığında ilgili favorite kaydının gerçekten o kullanıcıya ait olup olmadığı kontrol edilmelidir.

Benzer şekilde cart item işlemlerinde:

cart_item.user_id == current_user_id

kontrolü yapılmalıdır.

---

34. Resource Ownership

Authorization yalnızca role kontrolünden ibaret değildir.

Örneğin User A'nın cart item'ını User B değiştirememelidir.

Yanlış:

PATCH /api/cart/items/{cart_item_id}
↓
Update item

Doğru:

JWT
↓
current_user_id
↓
Find cart item
↓
Check cart_item.user_id
↓
Same user?
├── Yes → Update
└── No → 403

Aynı kontrol favorites gibi kullanıcıya özel kaynaklarda da uygulanacaktır.

---

35. Database Constraints

Authentication ve authorization güvenliğini desteklemek için database seviyesinde de constraint'ler kullanılacaktır.

Örneğin:

provider + provider_user_id → UNIQUE

Favorites:

user_id + recipe_id → UNIQUE

Cart için:

user_id + ingredient_id + unit

uygun şekilde unique hale getirilebilir.

Bu constraint'ler application logic'teki kontrollerin yanında ikinci bir güvenlik ve veri bütünlüğü katmanı oluşturur.

---

36. Test Strategy

Authentication ve Authorization için aşağıdaki testler oluşturulacaktır:

Authentication Tests

Valid Google token
Invalid Google token
Expired Google token
Invalid provider
Existing user login
New user creation
JWT generation
JWT expiration

Authorization Tests

Guest → protected endpoint
Authenticated user → user endpoint
Authenticated user → admin endpoint
Admin → admin endpoint
Invalid JWT
Expired JWT
Missing Authorization header
User accessing another user's resource

Ownership Tests

User A modifies User A cart item → allowed
User A modifies User B cart item → forbidden

User A removes User A favorite → allowed
User A removes User B favorite → forbidden

---

37. Authentication Error Handling

Backend authentication hataları standart API response formatına uygun olacaktır.

Örnek:

```
{
  "success": false,
  "error": {
    "code": "INVALID_TOKEN",
    "message": "The provided authentication token is invalid.",
    "details": {}
  }
}
```

Flutter bu hata kodlarını kendi `ApiException` / `Failure` yapısına dönüştürecektir.

UI'da backend'in teknik hata mesajının tamamı doğrudan kullanıcıya gösterilmemelidir.

Örneğin:

INVALID_TOKEN

için kullanıcıya:

Oturumunuz sona ermiş. Lütfen tekrar giriş yapın.

gibi kullanıcı dostu bir mesaj gösterilebilir.

---

38. MVP Authentication Scope

MVP authentication kapsamı:

Google Social Login
+
Backend Token Verification
+
Backend JWT
+
Secure Token Storage
+
Authenticated API Requests
+
Guest / User Separation
+
Admin Authorization
+
Resource Ownership
+

Logout

MVP dışında bırakılan özellikler:

Email / Password Login
Password Reset
Email Verification
Two-Factor Authentication
Refresh Token Rotation
Multiple Social Providers
Advanced Session Management
Device Management

Bu özellikler sistem büyüdüğünde değerlendirilebilir.

---

39. Future Authentication Extensions

İlerleyen sürümlerde aşağıdaki özellikler eklenebilir:

Google
Apple
Microsoft
Email / Password
Refresh Token
Token Rotation
2FA
Session Management
Account Linking

Yeni authentication provider eklendiğinde mevcut kullanıcı modeli:

provider
provider_user_id

alanları sayesinde genişletilebilir.

Örneğin:

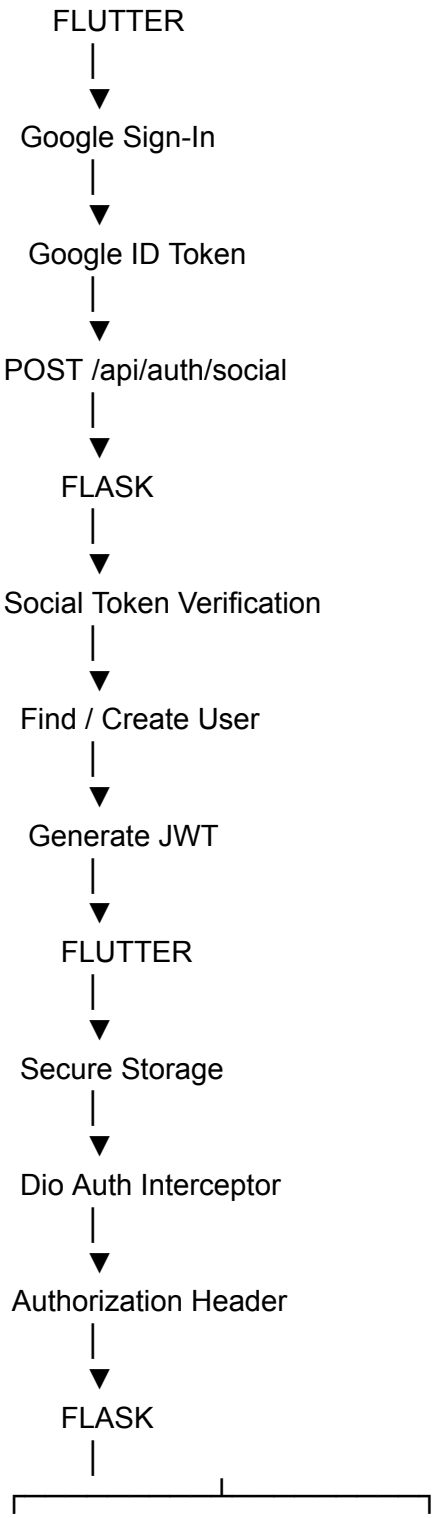
provider = google
provider = apple
provider = microsoft

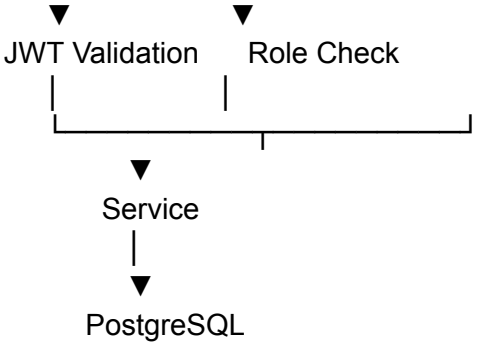
gibi farklı provider değerleri desteklenebilir.

---

40. Final Authentication Architecture

Smart Chef & Pantry AI authentication ve authorization mimarisi:





Guest User

Public API
↓
Ingredients
↓
Recipe Recommendation
↓
Recipe Detail
↓
External Source / YouTube

Registered User

Google Login
↓
Backend JWT
↓
Favorites
Cart
Profile
Protected APIs

Admin

Authenticated
+
Admin Role
↓
Admin API
↓
Recipe Management
Ingredient Management

---

41. Final Security Principles

Smart Chef & Pantry AI authentication sistemi aşağıdaki temel prensiplere göre geliştirilecektir:

1. Google token backend tarafından doğrulanacaktır.
2. Backend kendi JWT'sini oluşturacaktır.
3. JWT secret source code içerisinde tutulmayacaktır.
4. Tokenlar güvenli storage içerisinde saklanacaktır.
5. Protected endpoint'ler JWT gerektirecektir.
6. User ID request body'den alınmayacak, JWT üzerinden belirlenecektir.
7. Admin authorization backend'de yapılacaktır.
8. Kullanıcı kendi kaynakları dışındaki kaynaklara erişemeyecektir.
9. Token ve secret değerleri loglanmayacaktır.
10. Production ortamında HTTPS kullanılacaktır.
11. Authentication state Flutter tarafında merkezi olarak yönetilecektir.
12. Guest kullanıcı için public ve protected route ayrımı korunacaktır.
13. Authentication ile Authorization birbirinden ayrı sorumluluklar olarak uygulanacaktır.
14. API katmanı authentication detaylarını service katmanından ayıracaktır.
15. Authentication mimarisi ileride farklı provider ve refresh token mekanizmalarını destekleyecek şekilde tasarlanacaktır.

Bu yapı ile Smart Chef & Pantry AI'nin authentication ve authorization sistemi MVP için yeterli güvenlik ve mimari ayrımı sağlayacak, ilerleyen sürümlerde daha gelişmiş authentication özelliklerinin eklenmesine de uygun olacaktır.